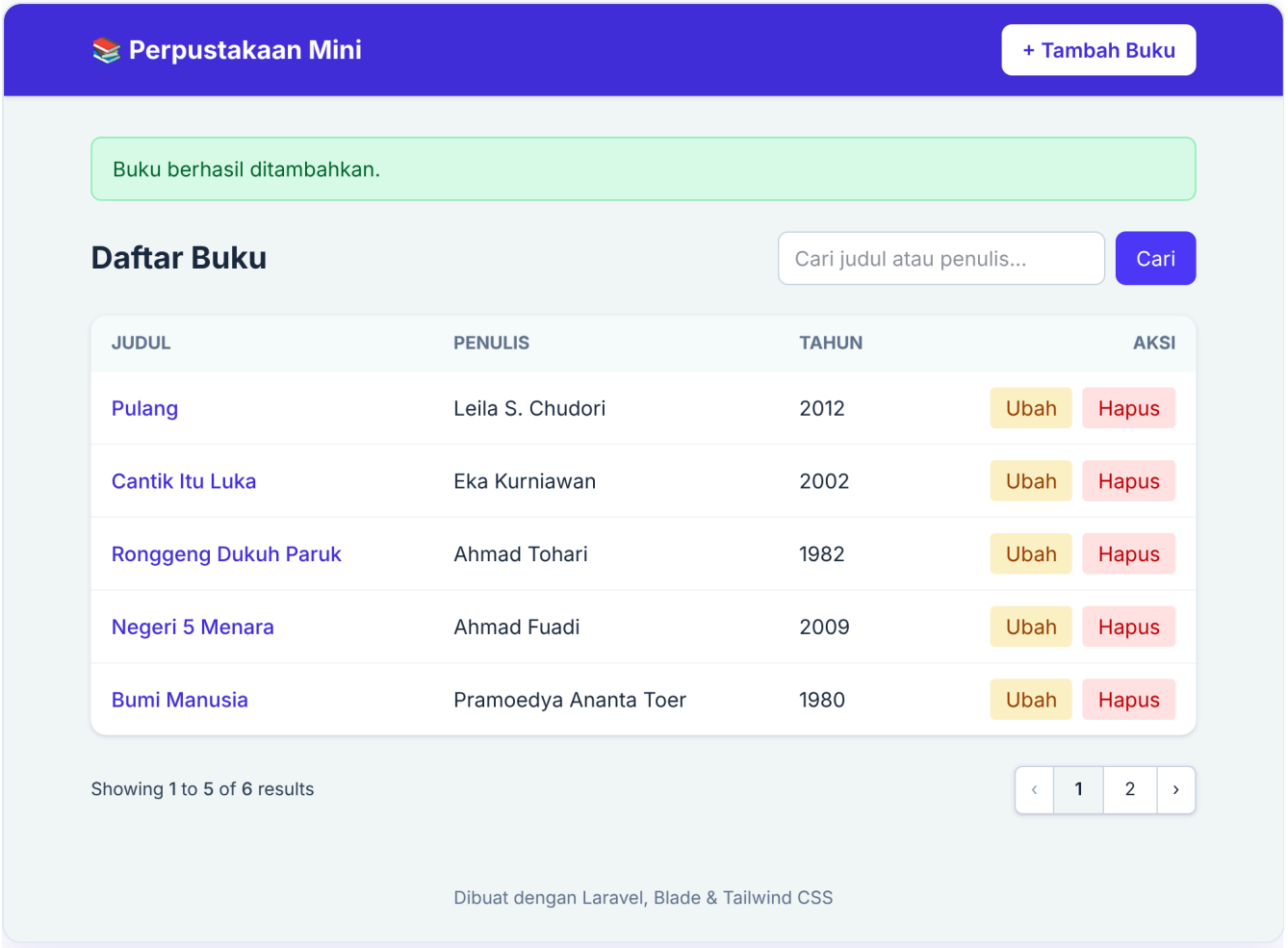
TUTORIAL LARAVEL UNTUK PEMULA

# Membangun Website Daftar Buku

Langkah demi langkah dengan **VS Code**, **Laravel**, **Blade**, dan **Tailwind CSS**




### Yang akan dibuat

Aplikasi "Perpustakaan Mini" untuk menambah, melihat, mengubah, menghapus, dan mencari data buku (CRUD), lengkap dengan validasi form dan pembagian halaman.

### Prasyarat

- VS Code sudah terpasang
- PHP, Composer, dan Node.js sudah terpasang (misalnya lewat Laragon atau Laravel Herd)
- Sudah pernah membuat proyek Laravel

### Perkiraan waktu

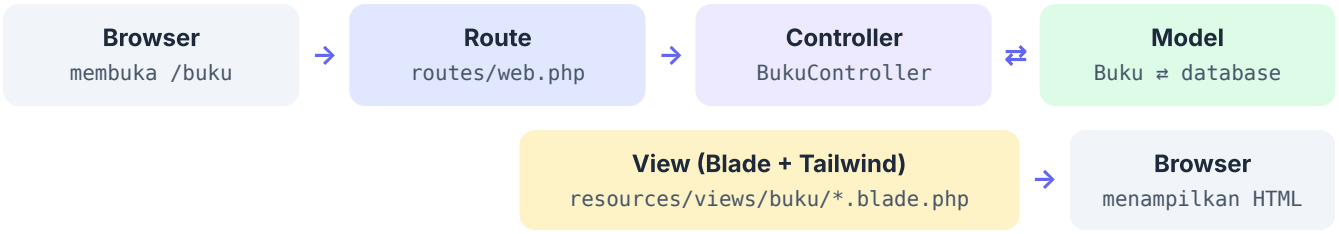
60–90 menit

### Versi

Laravel 12/13, Tailwind CSS 4

# Gambaran Besar: Bagaimana Laravel Bekerja

Laravel memakai pola **MVC (Model–View–Controller)**. Setiap kali pengunjung membuka sebuah alamat, permintaannya melewati beberapa "pos" berikut:




| Bagian | Tugas | Contoh di tutorial ini |
|---|---|---|
| **Route** | Menentukan alamat URL ditangani oleh siapa | `/buku` → `BukuController@index` |
| **Controller** | "Otak" yang mengatur alur: ambil data, validasi, simpan, lalu pilih tampilan | `BukuController.php` |
| **Model** | Perwakilan tabel database dalam bentuk class PHP | `Buku.php` ↔ tabel `buku` |
| **Migration** | "Cetak biru" struktur tabel, ditulis dalam PHP | kolom judul, penulis, tahun_terbit, sinopsis |
| **View (Blade)** | File HTML yang bisa diisi data dan logika sederhana | `index.blade.php`, `create.blade.php`, dst. |
| **Tailwind CSS** | Memberi gaya lewat class kecil di HTML | `bg-indigo-600 text-white rounded-lg` |

## Daftar file yang akan kita buat atau ubah

| File | Keterangan |
|---|---|
| `database/migrations/…_create_bukus_table.php` | Struktur tabel buku |
| `app/Models/Buku.php` | Model Buku |
| `app/Http/Controllers/BukuController.php` | Logika CRUD dan pencarian |
| `routes/web.php` | Daftar alamat URL |
| `resources/views/layouts/app.blade.php` | Kerangka halaman (menu, pesan, footer) |
| `resources/views/buku/index.blade.php` | Halaman daftar buku |
| `resources/views/buku/_form.blade.php` | Isian form yang dipakai bersama |
| `resources/views/buku/create.blade.php`, `edit.blade.php`, `show.blade.php` | Halaman tambah, ubah, dan detail |
| `database/seeders/BukuSeeder.php` | Data contoh |

## 1 Membuat Proyek Baru dan Membukanya di VS Code

Buka terminal (Command Prompt, PowerShell, atau terminal Laragon), masuk ke folder tempat Anda menyimpan proyek, lalu jalankan:

```
Terminal
$ composer create-project laravel/laravel daftar-buku
$ cd daftar-buku
$ code .
```

Perintah pertama mengunduh Laravel dan menyiapkan proyek bernama `daftar-buku`. Perintah `code .` membuka folder tersebut di VS Code.

> **Tips.** Pengguna Laragon dapat memakai menu **Quick app → Laravel** dan memberi nama `daftar-buku`. Pengguna Laravel Installer dapat menjalankan `laravel new daftar-buku` dan memilih **None** saat ditanya starter kit, karena kita akan membangun tampilan sendiri.

Mulai sekarang, semua perintah dijalankan di **terminal VS Code**: buka menu **Terminal → New Terminal** atau tekan Ctrl+`. Terminal ini otomatis berada di folder proyek.

### Kenali folder-folder penting

| Folder / File | Isi |
|---|---|
| `app/Http/Controllers` | Controller |
| `app/Models` | Model |
| `database/migrations` | Migration (struktur tabel) |
| `database/seeders` | Pengisi data contoh |
| `resources/views` | File Blade (tampilan) |
| `resources/css/app.css` | Titik masuk Tailwind CSS |
| `routes/web.php` | Daftar alamat URL |
| `.env` | Pengaturan lokal: nama aplikasi, koneksi database (jangan diunggah ke GitHub) |

## 2 Menyiapkan Database

Laravel versi baru memakai **SQLite** secara bawaan: database berupa satu file `database/database.sqlite` yang sudah dibuat otomatis saat proyek dibuat. Untuk belajar, ini cara paling mudah karena tidak perlu menyalakan MySQL. Buka file `.env` dan pastikan barisnya seperti ini:

```
.env
DB_CONNECTION=sqlite
```

### Alternatif: memakai MySQL di Laragon

Jika ingin memakai MySQL, nyalakan MySQL di Laragon, buat database kosong bernama `daftar_buku` (misalnya lewat HeidiSQL atau phpMyAdmin), lalu ubah `.env` menjadi:

.env

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=daftar_buku
DB_USERNAME=root
DB_PASSWORD=
```

Setelah itu jalankan `php artisan migrate` agar tabel bawaan Laravel (users, sessions, cache, dan lainnya) dibuat di MySQL.

> **Penting.** Hapus tanda `#` di depan baris DB_HOST sampai DB_PASSWORD; tanda itu membuat baris diabaikan. Di Laragon, password root MySQL biasanya kosong.

## 3 Menjalankan Server dan Tailwind (Vite)

Laravel membutuhkan dua proses yang berjalan bersamaan: **server PHP** untuk menjalankan aplikasi, dan **Vite** untuk memproses Tailwind CSS. Di VS Code, buka dua terminal (klik ikon **Split Terminal** di panel terminal).

Terminal 1 — server PHP

```
$ php artisan serve
```

Terminal 2 — Vite + Tailwind

```
$ npm install
$ npm run dev
```

Buka **http://127.0.0.1:8000** di browser. Jika halaman sambutan Laravel muncul, proyek sudah berjalan. Biarkan kedua terminal tetap terbuka selama Anda bekerja; buka terminal ketiga untuk menjalankan perintah lain.

> **Tips.** Cara singkat: `composer run dev` menjalankan server dan Vite sekaligus dalam satu terminal. `npm install` cukup dijalankan sekali saja di awal.

### Tailwind sudah siap pakai

Proyek Laravel baru sudah membawa Tailwind CSS 4. Anda tidak perlu memasang apa pun. Konfigurasinya ada di dua tempat:

resources/css/app.css

```
@import 'tailwindcss';

@source '../../vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php';
@source '../../storage/framework/views/*.php';

/* ...pengaturan font... */
```

vite.config.js (bagian penting)

```
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
    plugins: [
        laravel({
            input: ['resources/css/app.css', 'resources/js/app.js'],
```

```
            refresh: true,
        }),
        tailwindcss(),
    ],
});
```

Baris `@source '../../storage/framework/views/*.php'` membuat Tailwind memindai file Blade yang sudah dikompilasi, sehingga setiap class yang Anda tulis di file Blade otomatis dibuatkan CSS-nya. Di file Blade, CSS dimuat dengan direktif `@vite(...)`, yang akan kita tulis di Langkah 8.

> **Catatan.** Pasang ekstensi **Tailwind CSS IntelliSense** di VS Code agar muncul saran nama class saat mengetik.

## 4 Membuat Model, Migration, dan Controller

Satu perintah Artisan bisa membuat tiga file sekaligus:

Terminal VS Code

```
$ php artisan make:model Buku -mcr
```

| Opsi | Membuat | Lokasi |
|---|---|---|
| `-m` | Migration | `database/migrations/…_create_bukus_table.php` |
| `-c` | Controller | `app/Http/Controllers/BukuController.php` |
| `-r` | Controller berisi 7 method CRUD siap diisi | (digabung dengan `-c`) |

### Mengisi migration

Laravel memberi nama tabel dalam bentuk jamak bahasa Inggris, sehingga "Buku" menjadi `bukus`. Supaya rapi, kita ganti menjadi `buku`. Buka file migration yang baru dibuat dan ubah isinya menjadi:

database/migrations/xxxx_xx_xx_xxxxxx_create_bukus_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('buku', function (Blueprint $table) {
            $table->id();
            $table->string('judul');
            $table->string('penulis');
            $table->unsignedSmallInteger('tahun_terbit');
            $table->text('sinopsis')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('buku');
```

```
    }
};
```

Arti setiap kolom: `string` untuk teks pendek, `unsignedSmallInteger` untuk angka tahun, `text` untuk teks panjang, dan `nullable()` berarti boleh kosong. `id()` dan `timestamps()` otomatis menambahkan kolom `id`, `created_at`, dan `updated_at`.

Jalankan migration untuk membuat tabelnya:

**Terminal VS Code**

```
$ php artisan migrate
```

## 5 Mengatur Model Buku

Buka `app/Models/Buku.php` dan lengkapi seperti berikut:

**app/Models/Buku.php**

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Buku extends Model
{
    // Nama tabel di database (tanpa ini Laravel akan mencari "bukus")
    protected $table = 'buku';

    // Kolom yang boleh diisi lewat form
    protected $fillable = [
        'judul',
        'penulis',
        'tahun_terbit',
        'sinopsis',
    ];
}
```

- `$table` memberi tahu Laravel nama tabel yang benar (`buku`, bukan `bukus`).
- `$fillable` adalah daftar kolom yang boleh diisi sekaligus dari form. Ini fitur keamanan: kolom yang tidak tercantum akan diabaikan.

> **Catatan.** Di Laravel 13 Anda juga bisa menulis `#[Table('buku')]` dan `#[Fillable([...])]` di atas class. Cara di atas tetap berlaku dan berjalan di Laravel 12 maupun 13.

## 6 Mendaftarkan Route

Ganti seluruh isi `routes/web.php` menjadi:

**routes/web.php**

```php
<?php

use App\Http\Controllers\BukuController;
use Illuminate\Support\Facades\Route;
```

```
Route::redirect('/', '/buku');

Route::resource('buku', BukuController::class);
```

Satu baris `Route::resource` langsung membuat 7 alamat untuk CRUD:

| Method | URL | Method controller | Nama route | Fungsi |
|---|---|---|---|---|
| GET | `/buku` | index | `buku.index` | Daftar buku |
| GET | `/buku/create` | create | `buku.create` | Form tambah |
| POST | `/buku` | store | `buku.store` | Simpan data baru |
| GET | `/buku/{buku}` | show | `buku.show` | Detail |
| GET | `/buku/{buku}/edit` | edit | `buku.edit` | Form ubah |
| PUT | `/buku/{buku}` | update | `buku.update` | Simpan perubahan |
| DELETE | `/buku/{buku}` | destroy | `buku.destroy` | Hapus |

Cek hasilnya dengan `php artisan route:list --path=buku`. Baris `Route::redirect` membuat halaman utama ( `/` ) langsung diarahkan ke `/buku`.

## 7 Menulis Controller

Buka `app/Http/Controllers/BukuController.php` dan ganti isinya dengan kode berikut:

**app/Http/Controllers/BukuController.php**

```php
<?php

namespace App\Http\Controllers;

use App\Models\Buku;
use Illuminate\Http\Request;

class BukuController extends Controller
{
    // Menampilkan daftar buku (dengan pencarian)
    public function index(Request $request)
    {
        $cari = $request->query('cari');

        $daftarBuku = Buku::query()
            ->when($cari, function ($query, $cari) {
                $query->where('judul', 'like', "%{$cari}%")
                      ->orWhere('penulis', 'like', "%{$cari}%");
            })
            ->latest()
            ->paginate(5)
            ->withQueryString();

        return view('buku.index', compact('daftarBuku', 'cari'));
    }

    // Menampilkan form tambah buku
    public function create()
    {
        return view('buku.create');
    }
```

```
    // Menyimpan buku baru ke database
    public function store(Request $request)
    {
        $data = $request->validate($this->aturan());

        Buku::create($data);

        return redirect()->route('buku.index')
            ->with('sukses', 'Buku berhasil ditambahkan.');
    }

    // Menampilkan detail satu buku
    public function show(Buku $buku)
    {
        return view('buku.show', compact('buku'));
    }

    // Menampilkan form ubah buku
    public function edit(Buku $buku)
    {
        return view('buku.edit', compact('buku'));
    }

    // Menyimpan perubahan buku
    public function update(Request $request, Buku $buku)
    {
        $data = $request->validate($this->aturan());

        $buku->update($data);

        return redirect()->route('buku.index')
            ->with('sukses', 'Buku berhasil diperbarui.');
    }

    // Menghapus buku
    public function destroy(Buku $buku)
    {
        $buku->delete();

        return redirect()->route('buku.index')
            ->with('sukses', 'Buku berhasil dihapus.');
    }

    // Aturan validasi yang dipakai bersama oleh store() dan update()
    private function aturan(): array
    {
        return [
            'judul'        => 'required|string|max:255',
            'penulis'      => 'required|string|max:255',
            'tahun_terbit' => 'required|integer|min:1000|max:' . date('Y'),
            'sinopsis'     => 'nullable|string',
        ];
    }
}
```

### Penjelasan singkat

- **index()** mengambil data buku. `when($cari, …)` hanya menambahkan filter pencarian jika kotak cari diisi. `latest()` mengurutkan dari yang terbaru, `paginate(5)` membagi data menjadi 5 per halaman, dan `withQueryString()` menjaga kata kunci pencarian saat pindah halaman.
- **store()** dan **update()** memvalidasi data dengan `$request->validate()`. Jika gagal, Laravel otomatis kembali ke form beserta pesan errornya. Jika berhasil, data disimpan dan pengguna diarahkan ke

daftar buku dengan pesan sukses (`with('sukses', …)`).

- **show(Buku $buku)**, **edit()**, dan **destroy()** memakai *Route Model Binding*: Laravel otomatis mencari buku berdasarkan ID di URL. Jika tidak ditemukan, muncul halaman 404.
- `view('buku.index', compact(...))` berarti tampilkan file `resources/views/buku/index.blade.php` sambil mengirim variabel ke dalamnya.

## 8 Membuat Layout Utama dengan Blade

Layout adalah kerangka yang dipakai semua halaman: bagian `<head>`, menu navigasi, pesan sukses, dan footer. Buat folder `resources/views/layouts`, lalu buat file `app.blade.php` di dalamnya.

> **Catatan.** Di VS Code, klik kanan folder `resources/views` → **New File...**, lalu ketik `layouts/app.blade.php`. Folder `layouts` akan dibuat otomatis.

**resources/views/layouts/app.blade.php**

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('judul', 'Daftar Buku')</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class="min-h-screen bg-slate-100 text-slate-800">

    {{-- Navigasi --}}
    <nav class="bg-indigo-700 text-white shadow">
        <div class="max-w-4xl mx-auto px-4 py-4 flex items-center justify-between">
            <a href="{{ route('buku.index') }}" class="text-xl font-bold">📚 Perpustakaan Mini</a>
            <a href="{{ route('buku.create') }}"
               class="bg-white text-indigo-700 px-4 py-2 rounded-lg font-semibold hover:bg-indigo-50">
                + Tambah Buku
            </a>
        </div>
    </nav>

    <main class="max-w-4xl mx-auto px-4 py-8">

        {{-- Pesan sukses (flash message) --}}
        @if (session('sukses'))
            <div class="mb-6 rounded-lg bg-green-100 border border-green-300 text-green-800 px-4 py-3">
                {{ session('sukses') }}
            </div>
        @endif

        {{-- Isi halaman dari setiap view --}}
        @yield('konten')

    </main>

    <footer class="text-center text-sm text-slate-500 py-6">
        Dibuat dengan Laravel, Blade &amp;amp; Tailwind CSS
    </footer>

</body>
</html>
```

| Kode Blade | Fungsi |
|---|---|
| `@vite([...])` | Memuat CSS Tailwind dan JavaScript yang diproses Vite |
| `@yield('judul', 'Daftar Buku')` | Tempat kosong yang diisi oleh halaman lain; teks kedua adalah nilai bawaan |
| `@yield('konten')` | Tempat isi utama setiap halaman |
| `{{ route('buku.index') }}` | Menghasilkan URL dari nama route, jadi Anda tidak perlu menulis alamat secara manual |
| `@if (session('sukses'))` | Menampilkan pesan sukses yang dikirim controller |
| `{{-- ... --}}` | Komentar Blade (tidak tampil di browser) |

## 9 Halaman Daftar Buku

Buat folder `resources/views/buku`, lalu buat file `index.blade.php`:

**resources/views/buku/index.blade.php**

```
@extends('layouts.app')

@section('judul', 'Daftar Buku')

@section('konten')
    <div class="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4 mb-6">
        <h1 class="text-2xl font-bold">Daftar Buku</h1>

        {{-- Form pencarian --}}
        <form action="{{ route('buku.index') }}" method="GET" class="flex gap-2">
            <input type="text" name="cari" value="{{ $cari }}" placeholder="Cari judul atau penulis..."
                   class="w-64 rounded-lg border border-slate-300 px-3 py-2 focus:outline-none
focus:ring-2 focus:ring-indigo-500">
            <button class="bg-indigo-600 text-white px-4 py-2 rounded-lg hover:bg-indigo-
700">Cari</button>
        </form>
    </div>

    <div class="bg-white rounded-xl shadow overflow-hidden">
        <table class="w-full text-left">
            <thead class="bg-slate-50 text-sm uppercase text-slate-500">
                <tr>
                    <th class="px-4 py-3">Judul</th>
                    <th class="px-4 py-3">Penulis</th>
                    <th class="px-4 py-3">Tahun</th>
                    <th class="px-4 py-3 text-right">Aksi</th>
                </tr>
            </thead>
            <tbody class="divide-y divide-slate-100">
                @forelse ($daftarBuku as $buku)
                    <tr class="hover:bg-slate-50">
                        <td class="px-4 py-3 font-medium">
                            <a href="{{ route('buku.show', $buku) }}" class="text-indigo-700
hover:underline">
                                {{ $buku->judul }}
                            </a>
                        </td>
                        <td class="px-4 py-3">{{ $buku->penulis }}</td>
                        <td class="px-4 py-3">{{ $buku->tahun_terbit }}</td>
                        <td class="px-4 py-3">
                            <div class="flex justify-end gap-2">
```

```
                                <a href="{{ route('buku.edit', $buku) }}"
                                   class="px-3 py-1 rounded bg-amber-100 text-amber-800 hover:bg-amber-
200">Ubah</a>

                                <form action="{{ route('buku.destroy', $buku) }}" method="POST"
                                      onsubmit="return confirm('Yakin hapus buku ini?')">
                                    @csrf
                                    @method('DELETE')
                                    <button class="px-3 py-1 rounded bg-red-100 text-red-700 hover:bg-
red-200">Hapus</button>
                                </form>
                            </div>
                        </td>
                    </tr>
                @empty
                    <tr>
                        <td colspan="4" class="px-4 py-8 text-center text-slate-500">
                            Belum ada buku.
                        </td>
                    </tr>
                @endforelse
            </tbody>
        </table>
    </div>

    {{-- Navigasi halaman (pagination) --}}
    <div class="mt-6">
        {{ $daftarBuku->links() }}
    </div>
@endsection
```

### Yang perlu diperhatikan

- `@extends('layouts.app')` berarti halaman ini memakai layout dari Langkah 8. Isi di antara `@section('konten')` dan `@endsection` akan masuk ke `@yield('konten')`.
- `@forelse … @empty … @endforelse` seperti `@foreach`, tetapi punya bagian `@empty` yang tampil jika data kosong.
- Tombol hapus harus berupa **form**, karena link biasa hanya bisa mengirim GET. `@csrf` menambahkan token keamanan (wajib di setiap form POST), dan `@method('DELETE')` memberi tahu Laravel bahwa ini permintaan hapus.
- `{{ $daftarBuku->links() }}` menampilkan tombol halaman 1, 2, 3… yang sudah bergaya Tailwind secara otomatis.

> **Tips.** Perhatikan pola class Tailwind di tombol: `bg-…` warna latar, `text-…` warna teks, `px-/py-` jarak dalam, `rounded` sudut membulat, dan `hover:` gaya saat kursor di atasnya. Coba ganti `indigo` menjadi `emerald` atau `rose` untuk melihat perubahannya langsung.

## 10 Form yang Dipakai Bersama

Form tambah dan form ubah punya isian yang sama. Agar tidak menulis dua kali, kita buat satu file bagian (*partial*) lalu memanggilnya dengan `@include`. Buat file `resources/views/buku/_form.blade.php`:

**resources/views/buku/_form.blade.php**

```
{{-- Bagian form yang dipakai bersama oleh create dan edit --}}
<div class="space-y-5">
    <div>
```

```
        <label for="judul" class="block font-medium mb-1">Judul</label>
        <input type="text" id="judul" name="judul" value="{{ old('judul', $buku->judul ?? '') }}"
               class="w-full rounded-lg border px-3 py-2 @error('judul') border-red-500 @else border-
slate-300 @enderror">
        @error('judul')
            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
        @enderror
    </div>

    <div>
        <label for="penulis" class="block font-medium mb-1">Penulis</label>
        <input type="text" id="penulis" name="penulis" value="{{ old('penulis', $buku->penulis ?? '') }}"
               class="w-full rounded-lg border px-3 py-2 @error('penulis') border-red-500 @else border-
slate-300 @enderror">
        @error('penulis')
            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
        @enderror
    </div>

    <div>
        <label for="tahun_terbit" class="block font-medium mb-1">Tahun Terbit</label>
        <input type="number" id="tahun_terbit" name="tahun_terbit"
               value="{{ old('tahun_terbit', $buku->tahun_terbit ?? '') }}"
               class="w-40 rounded-lg border px-3 py-2 @error('tahun_terbit') border-red-500 @else
border-slate-300 @enderror">
        @error('tahun_terbit')
            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
        @enderror
    </div>

    <div>
        <label for="sinopsis" class="block font-medium mb-1">Sinopsis <span class="text-slate-400">
(opsional)</span></label>
        <textarea id="sinopsis" name="sinopsis" rows="4"
                  class="w-full rounded-lg border border-slate-300 px-3 py-2">{{ old('sinopsis', $buku-
>sinopsis ?? '') }}</textarea>
    </div>
</div>
```

- `old('judul', $buku->judul ?? '')` mengisi kembali isian dengan data yang tadi diketik jika validasi gagal. Jika tidak ada, ia memakai data buku (saat mengubah) atau kosong (saat menambah).
- `@error('judul') … @enderror` hanya tampil jika kolom tersebut gagal validasi. Variabel `$message` berisi pesan errornya.
- `@error('judul') border-red-500 @else border-slate-300 @enderror` mengganti warna garis input menjadi merah saat error, contoh nyata menggabungkan Blade dengan class Tailwind.

## 11 Halaman Tambah, Ubah, dan Detail

Ketiga halaman ini pendek karena memanfaatkan layout dan partial form.

**resources/views/buku/create.blade.php**

```
@extends('layouts.app')

@section('judul', 'Tambah Buku')

@section('konten')
    <div class="bg-white rounded-xl shadow p-6">
        <h1 class="text-2xl font-bold mb-6">Tambah Buku</h1>

        <form action="{{ route('buku.store') }}" method="POST">
```

```
            @csrf
            @include('buku._form')

            <div class="mt-6 flex gap-3">
                <button class="bg-indigo-600 text-white px-5 py-2 rounded-lg hover:bg-indigo-
700">Simpan</button>
                <a href="{{ route('buku.index') }}" class="px-5 py-2 rounded-lg border border-slate-300
hover:bg-slate-50">Batal</a>
            </div>
        </form>
    </div>
@endsection
```

**resources/views/buku/edit.blade.php**

```
@extends('layouts.app')

@section('judul', 'Ubah Buku')

@section('konten')
    <div class="bg-white rounded-xl shadow p-6">
        <h1 class="text-2xl font-bold mb-6">Ubah Buku</h1>

        <form action="{{ route('buku.update', $buku) }}" method="POST">
            @csrf
            @method('PUT')
            @include('buku._form')

            <div class="mt-6 flex gap-3">
                <button class="bg-indigo-600 text-white px-5 py-2 rounded-lg hover:bg-indigo-700">Simpan
Perubahan</button>
                <a href="{{ route('buku.index') }}" class="px-5 py-2 rounded-lg border border-slate-300
hover:bg-slate-50">Batal</a>
            </div>
        </form>
    </div>
@endsection
```

Perbedaan utama halaman ubah: form dikirim ke `route('buku.update', $buku)` dan memakai `@method('PUT')`, karena form HTML hanya mengenal GET dan POST.

**resources/views/buku/show.blade.php**

```
@extends('layouts.app')

@section('judul', $buku->judul)

@section('konten')
    <div class="bg-white rounded-xl shadow p-6">
        <h1 class="text-3xl font-bold">{{ $buku->judul }}</h1>
        <p class="text-slate-500 mt-1">{{ $buku->penulis }} &middot; {{ $buku->tahun_terbit }}</p>

        <div class="mt-6 text-slate-700 leading-relaxed">
            {{ $buku->sinopsis ?: 'Belum ada sinopsis.' }}
        </div>

        <div class="mt-8 flex gap-3">
            <a href="{{ route('buku.edit', $buku) }}" class="bg-amber-500 text-white px-4 py-2 rounded-lg
hover:bg-amber-600">Ubah</a>
            <a href="{{ route('buku.index') }}" class="px-4 py-2 rounded-lg border border-slate-300
hover:bg-slate-50">← Kembali</a>
        </div>
    </div>
@endsection
```

`$buku->sinopsis ?: 'Belum ada sinopsis.'` menampilkan teks pengganti jika sinopsis kosong.

## 12 Mengisi Data Contoh (Seeder)

Agar tidak perlu mengetik data satu per satu saat mencoba, buat seeder:

Terminal VS Code

```
$ php artisan make:seeder BukuSeeder
```

database/seeders/BukuSeeder.php

```
<?php

namespace Database\Seeders;

use App\Models\Buku;
use Illuminate\Database\Seeder;

class BukuSeeder extends Seeder
{
    public function run(): void
    {
        $data = [
            ['judul' => 'Laskar Pelangi', 'penulis' => 'Andrea Hirata', 'tahun_terbit' => 2005,
             'sinopsis' => 'Kisah sepuluh anak Belitung yang berjuang bersekolah di tengah
keterbatasan.'],
            ['judul' => 'Bumi Manusia', 'penulis' => 'Pramoedya Ananta Toer', 'tahun_terbit' => 1980,
             'sinopsis' => 'Kisah Minke, pemuda pribumi pada masa kolonial Hindia Belanda.'],
            ['judul' => 'Negeri 5 Menara', 'penulis' => 'Ahmad Fuadi', 'tahun_terbit' => 2009,
             'sinopsis' => 'Perjalanan Alif dan lima sahabatnya menuntut ilmu di sebuah pesantren.'],
            ['judul' => 'Ronggeng Dukuh Paruk', 'penulis' => 'Ahmad Tohari', 'tahun_terbit' => 1982,
             'sinopsis' => 'Kisah Srintil, penari ronggeng dari sebuah dukuh kecil di Banyumas.'],
            ['judul' => 'Cantik Itu Luka', 'penulis' => 'Eka Kurniawan', 'tahun_terbit' => 2002,
             'sinopsis' => 'Saga keluarga yang membentang dari masa kolonial hingga pascakemerdekaan.'],
            ['judul' => 'Pulang', 'penulis' => 'Leila S. Chudori', 'tahun_terbit' => 2012,
             'sinopsis' => 'Kisah para eksil politik Indonesia di Paris dan kerinduan akan tanah air.'],
        ];

        foreach ($data as $buku) {
            Buku::create($buku);
        }
    }
}
```

Jalankan seeder tersebut:

Terminal VS Code

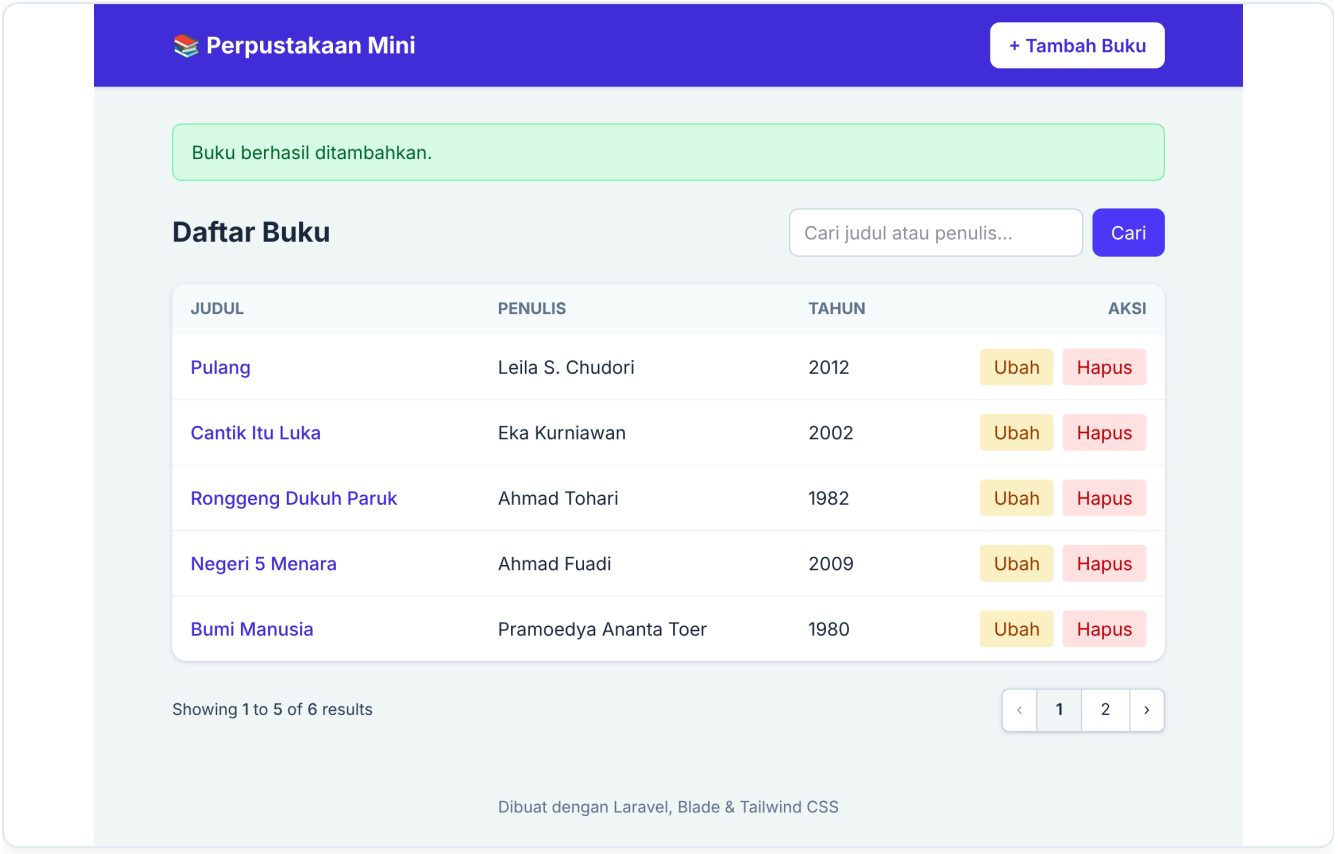
```
$ php artisan db:seed --class=BukuSeeder
```

**Penting.** Ingin mengulang dari nol? `php artisan migrate:fresh --seed` menghapus semua tabel, membuatnya ulang, lalu menjalankan `DatabaseSeeder`. Agar BukuSeeder ikut berjalan, tambahkan `$this->call(BukuSeeder::class);` di dalam method `run()` pada `DatabaseSeeder.php`. Perintah ini menghapus semua data, jadi gunakan hanya saat belajar.

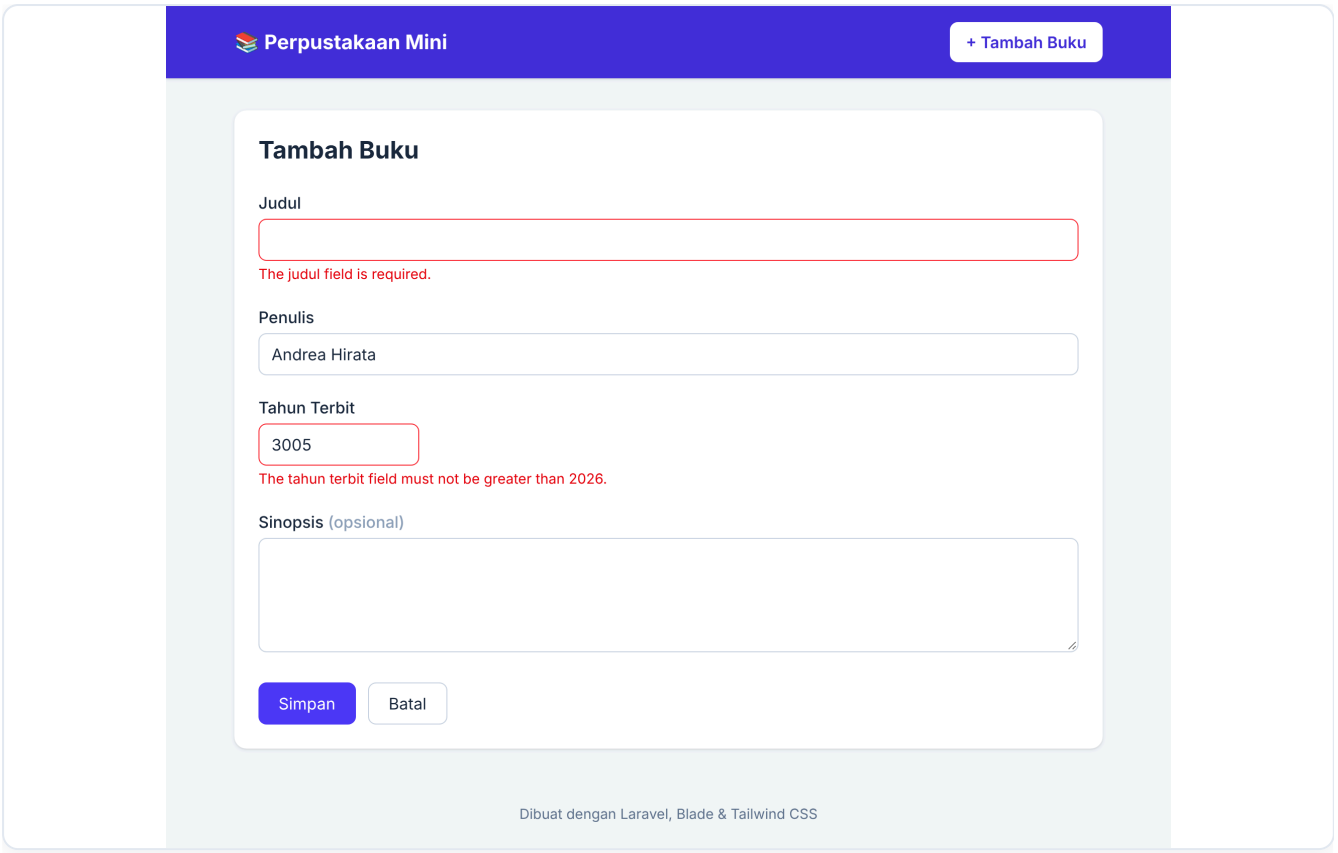
## 13 Mencoba Hasilnya

Pastikan `php artisan serve` dan `npm run dev` masih berjalan, lalu buka **http://127.0.0.1:8000**. Coba satu per satu:

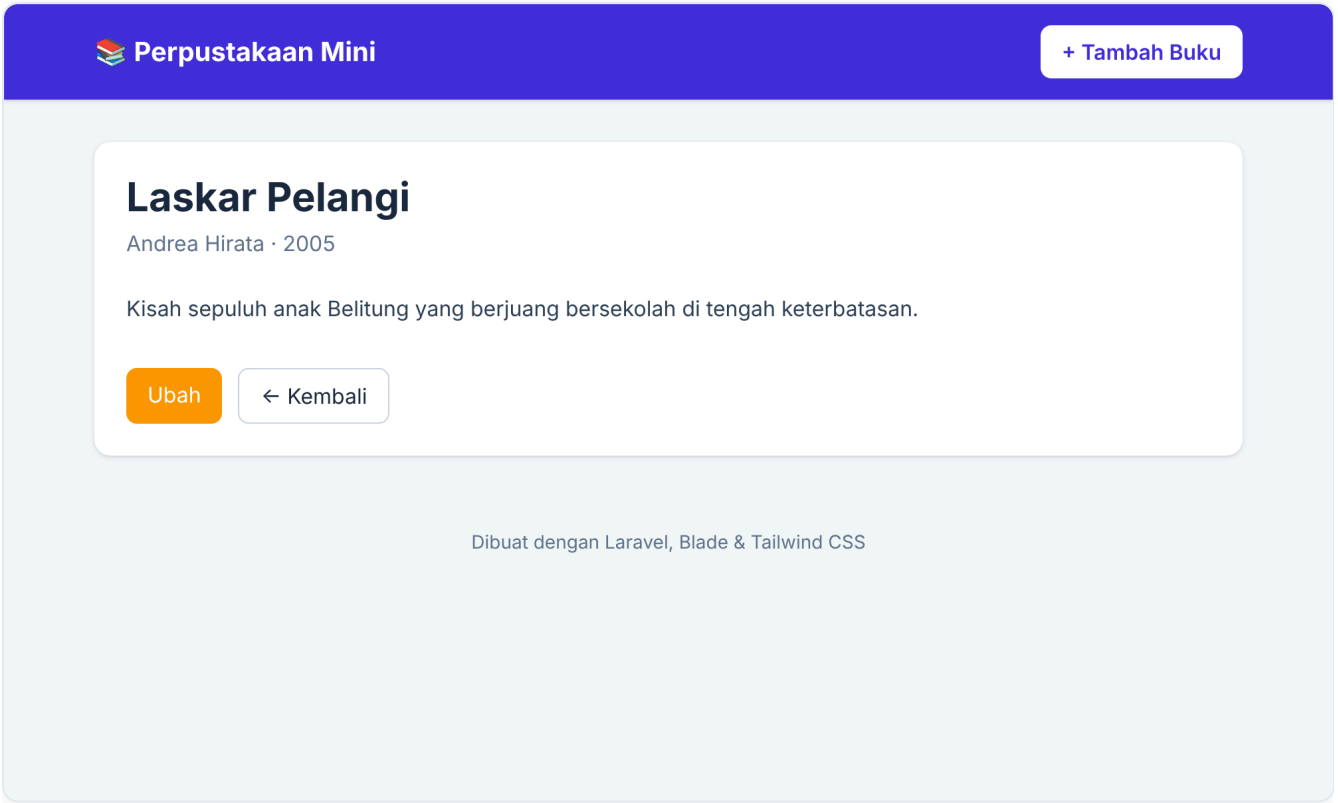
1. Halaman daftar menampilkan 5 buku dan tombol halaman 2 untuk buku ke-6.
2. Ketik "Ahmad" di kotak cari: hanya buku karya Ahmad Tohari dan Ahmad Fuadi yang tampil.
3. Klik **+ Tambah Buku**, kosongkan judul, isi tahun 3005, lalu klik Simpan: muncul pesan error berwarna merah dan isian penulis tidak hilang.
4. Isi form dengan benar lalu simpan: muncul pesan hijau "Buku berhasil ditambahkan."
5. Klik judul buku untuk melihat detail, lalu klik **Ubah** dan simpan perubahan.
6. Klik **Hapus**, konfirmasi, dan buku hilang dari daftar.




Halaman daftar buku dengan pesan sukses, pencarian, dan pagination




Form tambah buku saat validasi gagal: garis merah dan pesan error di bawah isian




Halaman detail buku

**Catatan.** Pesan error validasi tampil dalam bahasa Inggris karena itu bahasa bawaan Laravel. Cara menerjemahkannya dibahas di bagian Latihan.

## 14 Menyimpan Proyek ke GitHub dari VS Code

1. Klik ikon **Source Control** di sidebar kiri (Ctrl + Shift + G).
2. Klik **Publish to GitHub**, pilih *private* atau *public*.
3. Pastikan file `.env`, folder `vendor`, dan `node_modules` **tidak** ada di daftar unggahan. Laravel sudah mengaturnya lewat `.gitignore`.
4. Setiap selesai satu langkah tutorial, tulis pesan commit (misalnya "Tambah halaman daftar buku"), klik **Commit**, lalu **Sync Changes**.

## Jika Terjadi Error

| Pesan / gejala | Penyebab | Solusi |
|---|---|---|
| `Vite manifest not found` | Vite belum berjalan | Jalankan `npm install` lalu `npm run dev` |
| Tampilan polos, tanpa warna | CSS belum dimuat | Pastikan ada `@vite([...])` di layout dan `npm run dev` berjalan |
| `no such table: buku` / `Table … doesn't exist` | Migration belum dijalankan atau nama tabel tidak cocok | Jalankan `php artisan migrate`; cek `$table = 'buku'` di model sama dengan nama di migration |
| `419 Page Expired` | Lupa token CSRF | Tambahkan `@csrf` di dalam setiap `<form method="POST">` |
| `Add [judul] to fillable property…` | Kolom belum diizinkan | Tambahkan nama kolom ke `$fillable` di model |
| `View [buku.index] not found` | Nama atau lokasi file salah | File harus `resources/views/buku/index.blade.php` (akhiran `.blade.php`) |
| `Route [buku.index] not defined` | Route belum terdaftar | Cek `routes/web.php`, lalu `php artisan route:list` |
| `Method Not Allowed` saat ubah/hapus | Lupa `@method` | Tambahkan `@method('PUT')` atau `@method('DELETE')` |
| Class Tailwind baru tidak berefek | Vite belum memindai ulang | Simpan file, muat ulang browser; jika perlu hentikan dan jalankan lagi `npm run dev` |

## Latihan Lanjutan

1. **Tambah kolom baru.** Tambahkan kolom `penerbit`: buat migration baru dengan `php artisan make:migration add_penerbit_to_buku_table`, tambahkan ke `$fillable`, aturan validasi, form, dan tabel daftar.
2. **Pesan error berbahasa Indonesia.** Ubah `APP_LOCALE=id` di `.env`, jalankan `php artisan lang:publish`, salin folder `lang/en` menjadi `lang/id`, lalu terjemahkan isi `lang/id/validation.php`. Alternatif lain: kirim pesan sendiri sebagai argumen kedua `$request->validate($aturan, ['judul.required' => 'Judul wajib diisi.'])`.
3. **Perindah tampilan.** Ubah tabel menjadi kartu (*card*) dengan `grid grid-cols-1 md:grid-cols-2 gap-4`, lalu coba di layar HP dengan fitur *responsive mode* di browser (F12).
4. **Tambah login.** Pelajari starter kit Laravel atau paket `laravel/breeze` agar hanya pengguna yang login yang bisa menambah dan menghapus buku.

## Ringkasan Perintah

| Perintah | Fungsi |
|---|---|
| `composer create-project laravel/laravel nama` | Membuat proyek baru |
| `php artisan serve` | Menjalankan server lokal |
| `npm install` / `npm run dev` | Memasang dan menjalankan Vite + Tailwind |
| `composer run dev` | Server dan Vite sekaligus |
| `php artisan make:model Nama -mcr` | Model + migration + resource controller |
| `php artisan make:seeder NamaSeeder` | Membuat seeder |
| `php artisan migrate` | Menjalankan migration |
| `php artisan migrate:fresh --seed` | Mengulang database dari nol + data contoh |
| `php artisan db:seed --class=NamaSeeder` | Menjalankan satu seeder |
| `php artisan route:list` | Melihat semua route |
| `npm run build` | Membuat CSS/JS versi final untuk hosting |

Selamat! Anda telah membangun aplikasi CRUD lengkap dengan Laravel, Blade, dan Tailwind CSS.
Pola yang sama (migration → model → route → controller → view) dapat dipakai untuk aplikasi apa pun: data pegawai, inventaris, pendaftaran, dan lainnya.